

Final Project Report

Authors:

Bobby Taing

Date: 08/22/2025

EE/CSE 474 Summer 2025

Introduction	2
System Design and Implementation	2
Code Structure	3
Discussion	4
Division of Work	8
Suggestions	9
Conclusions	9
References	10
Appendices	11
Total Number of hours Spent	11

Introduction

The final project focuses on developing an automated plant watering system using the ESP32-S3. The primary programming challenges include integrating multiple peripherals: soil moisture sensor, water pump, relays and ultrasonic sensor. In addition, the system should maintain real-time responsiveness through dual-core scheduling, hardware timers, and queue-based communication. The key difficulty lies in coordinating time-critical pump control with LCD updates without interference. This required implementing precise periodic timers, debounced button handling, and reliable state management. By using two cores to handle control and display tasks separately, the system avoids jitter from slower I2C transactions and ensures stable operation. Learning objectives include strengthening skills learned in previous labs: interrupt, multicore task management with FreeRTOS, debouncing, Interrupt-Service-Routine (ISR) signalling, and peripheral interfacing. By combining all the skills, one can build a project that applies embedded systems principles to a practical, real-world application. For this project, the system tracks readings from the sensor, displays them on the LCD, and automatically waters the plant when it becomes dry. This allows the user to work on other tasks without worrying about forgetting to water the plant.

System Design and Implementation

The system was designed around the goal of reliable real-time control while integrating multiple hardware peripherals. To achieve this, the ESP32-S3's dual-core architecture and FreeRTOS framework were used to separate time-critical pump control from user interface rendering. The implementation began with configuring two high-resolution hardware timers. The 50 Hz timer generates control ticks that pace the pump logic, ensuring soil moisture and ultrasonic measurements are processed at stable intervals. A 64 Hz timer drives the LCD task, enabling frequent but efficient screen updates without overloading the I2C bus. An ISR is beneficial to post a simple flag into a FreeRTOS queue, deferring the work to tasks without waiting for specific tasks to be ready. This approach minimizes ISR latency and keeps the system responsive.

Two tasks are defined on separate cores of the system:

- TaskPump (Core 1, higher priority): handles soil moisture readings, ultrasonic tank measurements, button debouncing, and the pump state. The control logic enforces the pump to water for a period of time, waits for water to seep into the soil, and automatically suspends if the water tank is empty.
- TaskLCD (Core 0, lower priority): updates the I2C LCD with soil moisture, elapsed idle time, tank distance, and pump status. The readings on the LCD redraw only when values from the sensors change, which reduces flicker.

This design decouples control and display, preventing I2C transactions from interfering with pump responsiveness. Debounce buttons are added for manual watering and suspension, and an LED to indicate an empty water tank or that the pump is suspended. By combining precise timers, dual-core task separation, and queue-based synchronization, the system achieves deterministic timing, stable operation, and responsiveness to both environmental conditions and user input.

Code Structure

The code is organized into modular components that separate hardware interaction, control logic, and user interface responsibilities. This structure improves clarity and maintainability, ensuring that time-critical operations remain isolated from slower tasks.

The main modules are:

- Timer Configuration and ISR Module: defines two high-resolution timers (50 Hz and 64 Hz). Each timer callback is lightweight, posting a flag to a FreeRTOS queue to signal the corresponding task. This design avoids heavy computation inside ISRs and ensures deterministic timing.
- TaskPump (Control Module): runs on Core 1 with higher priority. It performs sensor sampling, button debouncing, and pump state management. Logic here enforces a minimum ON for ten seconds when the soil moisture is under a 30% threshold and waits for ninety seconds for the water to seep into the soil before checking again. Two buttons are defined: one for manual watering and the other for suspending the water pump. The LED indicator is implemented to turn ON when the sensor detects an empty tank or the pump is shut down. Finally, the module publishes system snapshots to qStatus for the LCD task.
- TaskLCD (UI Module): runs on Core 0 with lower priority. It receives snapshots from qStatus and refreshes the LCD at 64 Hz. Redraw minimization ensures that updates occur only when values change, reducing flicker and I2C overhead. The UI displays soil moisture, elapsed idle time, tank water level, and pump ON/OFF status with custom icons.
- State Management Functions: utility functions handle sensor conversions, such as `moisturePercentFromRaw()` for mapping ADC values to percentages and `PingUltrasonicCM()` for safe ultrasonic distance measurement with timeout protection. These encapsulate sensor logic and provide consistent, bounded outputs for the control module.

By organizing the code into timers, tasks, and state management modules, the system cleanly separates responsibilities. This ensures that each component contributes to overall functionality while maintaining responsiveness and robustness.

Discussion

The final setup and testing of the automated plant watering system are illustrated below. Each figure demonstrates a different mode of operation or condition handled by the system.

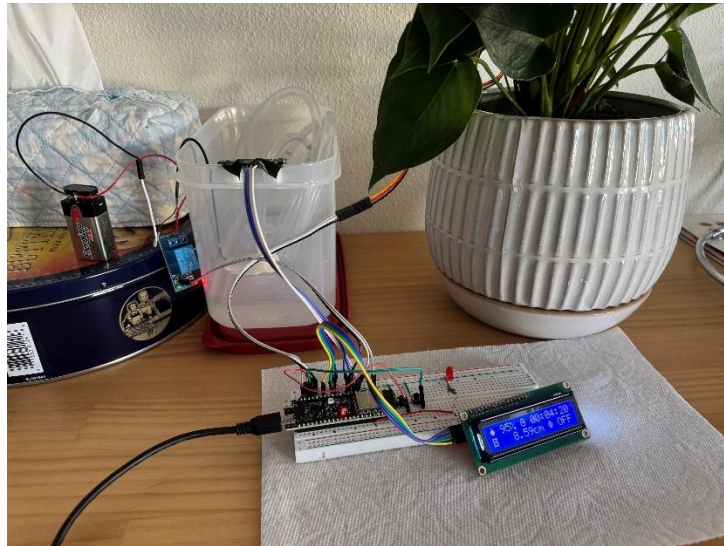


Figure 1: System Setup

Figure 1 shows the complete hardware configuration, including the ESP32-S3, capacitive soil moisture sensor, ultrasonic sensor, relay-driven pump, LCD, LED indicator, and push buttons for manual watering (green wire) and suppression (red wire). The LCD displays live soil moisture data, elapsed idle time, tank level, and pump status. Moisture reading is at 95% which makes sense since the pot was on the edge of overflowing, and the distance from the tank shows that it is not empty. The components are integrated to provide real-time sensing, control, and user feedback.

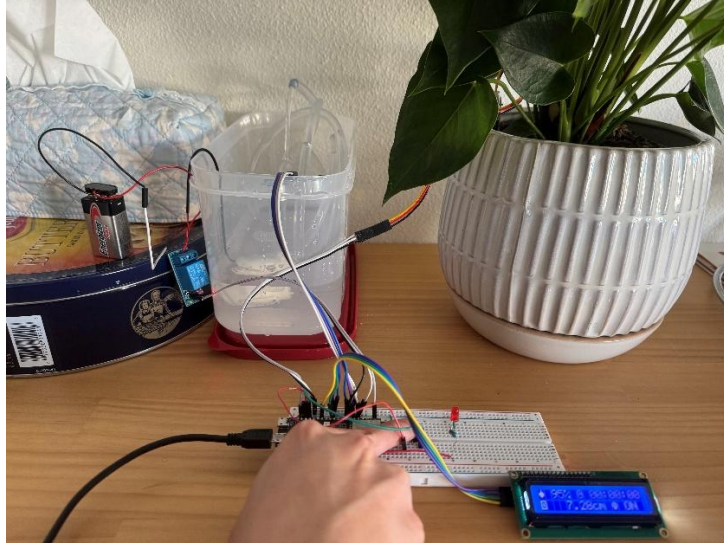


Figure 2: Manual Watering

Figure 2 shows that the manual button is pressed, activating the relay and turning on the pump seen from the end of the tube. The suppress button was not pressed, and the threshold distance is valid.

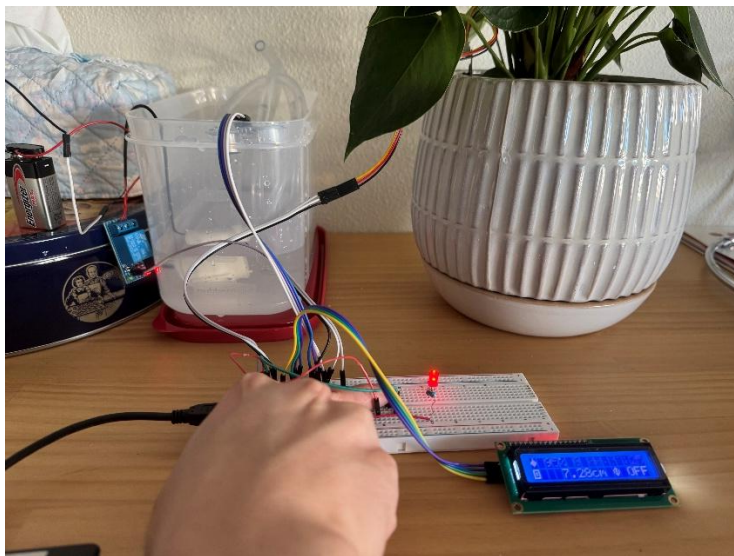


Figure 3: Manual Watering with Suppression Enabled

Figure 3 shows the suppression buttons pressed before manual watering. The suppression LED turns on, and the pump is disabled despite manual input. This verifies that the suppression state correctly overrides the manual pump command.



Figure 4: Manual Watering with Low Tank Warning

Figure 4 shows the ultrasonic sensor placed outside the container to verify that the distance it detects is towards the table. The LED indicator turns on when it is more than 10 cm. Even though the manual button is pressed, the system prioritizes safety by preventing pump operation when the tank is nearly empty.

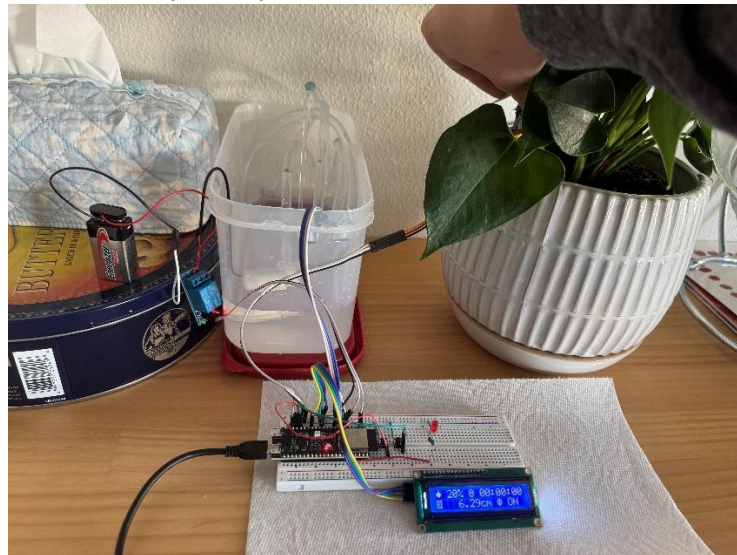


Figure 5: Automatic Watering

Figure 5 shows automatic watering triggered when soil moisture falls below the threshold (30%) via partially pulling the sensor off the soil. The pump activates for 10 seconds and then waits for 90 seconds before checking if it needs watering. The LCD updates in real-time, confirming that the autonomous control logic operates as designed.

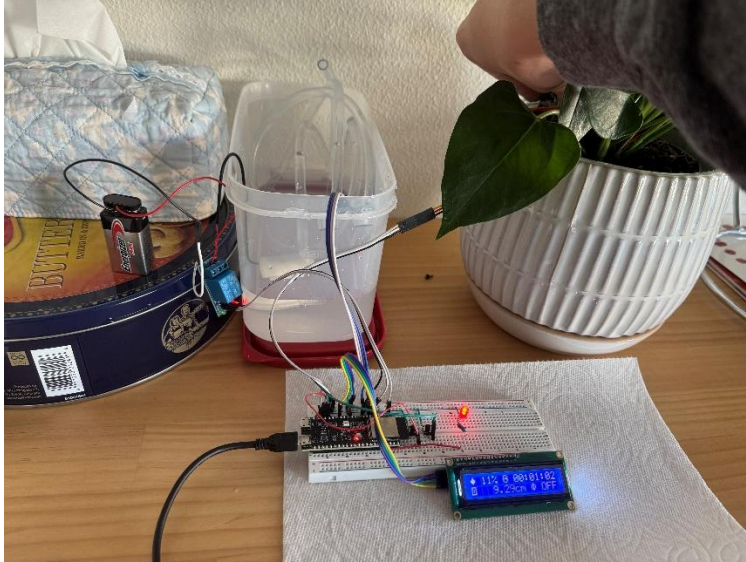


Figure 6: Automatic Watering with Suppression Enabled

Figure 6 shows that suppression is toggled while the soil is below the threshold. The LED indicates suppression is active, and the pump is forcibly turned off. This demonstrates that suppression reliably overrides auto control logic.

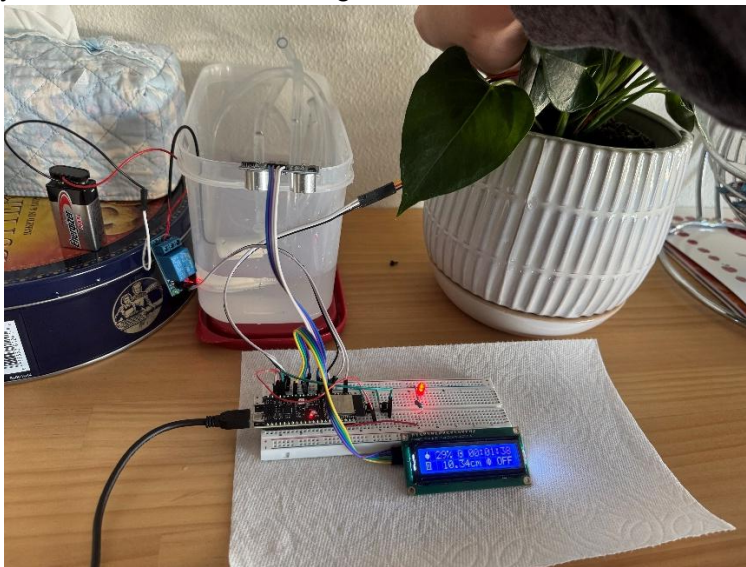


Figure 7: Automatic Watering with Low Tank Warning

Figure 7 shows that the ultrasonic sensor indicates the water tank is empty during auto watering. The LED turns on, and the pump is immediately disabled to prevent dry running. This confirms that safety constraints are enforced under all operating modes.

This project presented several challenges that tested both my hardware integration and software debugging skills. One of the first issues was correctly wiring the pump relay to the ESP32-S3. Initially, connecting to the relay to 5V caused the pump to remain permanently on. Through experimentation and documentation review, I realized the relay must be driven at 3.3V

logic to properly switch states properly, ensuring the pump could be reliably controlled by the microcontroller.

Sensor selection also created difficulties. Early tests with a resistive soil moisture probe showed highly inconsistent readings that fluctuated due to soil composition and corrosion effects. To resolve this, I transitioned to a capacitive sensor, which produced more stable values and integrated cleanly with the ADC mapping functions.

Midway through development, I encountered a major hardware setback. Water was accidentally spilled on the ESP32-S3, which was then powered by a 9V battery pack, rendering it permanently unusable. To avoid halting progress, I adapted the project to run on an Arduino Mega 2560 for several days over the weekend. This presented its own challenges, since the Mega lacks dual-core support, and task responsiveness suffered as a result. Additionally, I had to modify the code to match the Mega's pin mappings and limited resources. While functionality was reduced, this stopgap solution allowed me to continue testing for other functionalities.

The biggest challenge, which had the most setbacks, was the delay in receiving ordered components. While waiting for deliveries, I leveraged hardware I already had, which included the LCD, buttons, and LEDs, to prototype core features like manual pump control, suppression toggling, and custom icon rendering. This incremental approach allowed me to validate parts of the system even before the full hardware stack was assembled.

This was an independent project; I worked entirely on my own, managing both hardware and software integration. While this required additional effort, it provided valuable experience in troubleshooting across domains from debugging hardware wiring errors to restructuring software for different microcontrollers.

These challenges reinforced key lessons: the importance of choosing reliable sensors, the need for fail-safe hardware design, and the value of modular code that can be adapted across platforms. Despite setbacks, each difficulty provided an opportunity to refine both my technical understanding and project resilience.

Division of Work

This project was completed independently. I was 100% responsible for all aspects of the design, implementation, testing, and reporting. This included hardware integration, software development, debugging across both ESP32-S3 and Arduino Mega platforms, as well as documentation and final writing.

Suggestions

For the project, one significant enhancement would be to integrate Wi-Fi connectivity. This would allow remote monitoring and control, enabling users abroad to verify system status and intervene, if necessary, such as shutting down the pump. Additional sensors for temperature, humidity, and light could provide a more complete environmental profile, allowing the system to make more intelligent watering decisions. Expanding the user interface to include data logging or mobile app integration would further align the system with IoT applications.

From an instructional standpoint, providing students with a head start would inspire ideas and encourage early brainstorming, such as video examples in the second week of the quarter. Since the final project depends on hardware that is not always provided in class, students would benefit from guidance on ordering components early to avoid delays in integration. Supplementary references on new hardware peripherals (a popular choice) would provide students with a basic technical background before selecting parts. It may also be helpful to add safety cautions when suggesting peripherals outside the course scope, so students are aware of potential risks with electrical loads or water-based experiments.

Conclusions

This project successfully implemented an automated plant watering system on the ESP32-S3, meeting the objectives of reliable timing, dual-core task management, and robust sensor integration. By combining capacitive soil moisture sensing, ultrasonic water level detection, an LCD interface, and relay-based pump control, the system operated responsively and safely under real-world conditions.

The use of hardware timers, ISRs, and FreeRTOS queues ensured deterministic task execution, while separating control and display tasks across cores prevented interference. This experience strengthened my skills in interrupt-driven design, multicore programming, and embedded systems integration, building directly on concepts from earlier labs.

Overall, the project provided valuable practice in applying embedded systems theory to a practical application, reinforcing both technical and problem-solving skills.

References

- [1] Dejan Nedelkovski, "Ultrasonic Sensor HC-SR04 and Arduino Tutorial," *HowToMechatronics*. Available: <https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/>
- [2] Robotique Tech, "Control a Water Pump by Arduino," *Robotique Tech*. Available: <https://www.robotique.tech/robotics/control-a-water-pump-by-arduino/>
- [3] Last Minute Engineers, "I²C LCD with Arduino Tutorial," *LastMinuteEngineers*. Available: <https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/>
- [4] Espressif Systems, "Memory Types," *ESP-IDF Programming Guide*, 2024. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/memory-types.html>
- [5] Espressif Systems, "FreeRTOS Overview," *ESP-IDF Programming Guide*, 2024. Available: <https://docs.espressif.com/projects/esp-idf/en/v4.4.2/esp32/api-reference/system/freertos.html>
- [6] Espressif Systems, "ADC – Analog to Digital Converter API," *Arduino-ESP32 Documentation*, 2024. Available: <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html>
- [7] Espressif Systems, "High Resolution Timer (esp_timer)," *ESP-IDF Programming Guide*, 2024. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html
- [8] ESP32 Tutorials, "High Resolution Timer – esp_timer in ESP-IDF," *ESP32 Tutorials*. Available: <https://esp32tutorials.com/high-resolution-timer-esp-timer-esp-idf/>
- [9] ESP32 Tutorials, "FreeRTOS Timer and Delay in ESP-IDF," *ESP32 Tutorials*. Available: <https://esp32tutorials.com/esp32-esp-idf-freertos-timer-delay-esp-idf/>

Appendices

Total Number of hours Spent

- Hardware debugging: 6 hours
- Software debugging: 12 hours
- Doxygen: 1 hour
- Commenting: 2 hours
- Lab Report: 4 hours
 - o Total hours: **25 hours**